Crack Detection in Water Infrastructure with YOLO

Junhui Woo

December 10, 2024

Abstract

This project aims to automate the detection of cracks in large and inaccessible water resource facilities, such as dams and wastewater treatment plants, using artificial intelligence. Specifically, I employ the YOLO (You Only Look Once) object detection framework to identify objects in images in real time with high accuracy and efficiency. By leveraging YOLO's capabilities, I aim to improve the monitoring and inspection processes of these critical structures, reducing the risk of safety hazards associated with undetected cracks. In this project, crack data was trained using YOLOv5, the results were analyzed, and tests were conducted to evaluate the model's ability to detect cracks.

1 Introduction

Water facilities encompass a variety of structures and systems designed for the efficient management, supply, and treatment of water resources. These facilities are essential for public safety and environmental protection. Major types include dams, weirs, water treatment plants, wastewater treatment plants, reservoirs, and irrigation systems.

Typically, water facilities are located over vast areas and often in rugged terrain or remote locations, making them difficult for many people to access. Additionally, these facilities are structurally large and complex, making it challenging to identify anomalies such as cracks or obstacles. When inspections are conducted manually, there are inherent risks, and visibility may be limited in certain terrains, which hampers the efficiency of regular monitoring.

(a) Dam

(b) Water Treatment Plant

Figure 1: Water facilities

Various attempts have been made to validate water facilities, but traditional inspection methods come with several limitations. For example, relying on visual inspections can introduce subjective judgment, making precise observations difficult. These inspections are also time-consuming and can pose safety risks, especially in hazardous environments. To overcome these challenges, small mechanical devices like drones and mobile robots can be employed to automate inspections across large areas without time constraints. For drones and mobile robots to efficiently inspect facilities, they must be capable of identifying structural constraints and obstacles, navigating accordingly, and transmitting data to quickly address anomalies such as

cracks. Consequently, drones and mobile robots have been the focus of active research in the field of computer vision over the past decade, aimed at processing image data quickly and accurately.

(a) Mobile Robot (b) Drone

Figure 2: Robots and drones equipped with cameras.

Convolutional Neural Networks (CNNs) have seen significant advancements over the years, leading to the development of efficient object detection models like YOLO (You Only Look Once). In 1998, LeNet [1] introduced CNN for handwritten digit recognition. It employed convolution and pooling layers to process images effectively, but it was limited to small datasets and simple images. The next major breakthrough came in 2012 with AlexNet [2], which won the ImageNet competition. AlexNet used ReLU activation and dropout, enabling large-scale image training with GPUs. However, it still faced difficulties in recognizing variations in object size and location.

In 2014, VGGNet [3] was developed, increasing the depth of networks using small 3x3 filters. However, this came at the cost of higher computational requirements, making it unsuitable for real-time applications. That same year, GoogLeNet (Inception) [4] introduced an efficient design using multiple filter sizes simultaneously, which improved performance and efficiency, though its complex structure made implementation more challenging. In 2015, ResNet [5] addressed the vanishing gradient problem by introducing residual connections, allowing for much deeper networks. While ResNet was effective for image classification, it did not directly address real-time object detection.

Simultaneously, region-based approaches emerged for object detection. In 2014, RCNN [6] applied CNNs to object detection by generating region proposals for each image. However, the model was slow because each proposal had to be processed independently. Fast RCNN [7] improved this by extracting features once and processing the proposals afterward, though it still relied on external region proposals. In 2015, Faster RCNN [8] introduced the Region Proposal Network (RPN), which generated region proposals within the CNN itself, significantly improving speed, but still not reaching real-time performance.

YOLO [9], introduced in 2016, revolutionized object detection by framing it as a single regression problem. Unlike previous models, YOLO processed the entire image in one pass, simultaneously predicting object locations and classes, allowing for real-time detection. However, YOLO struggled with detecting small or overlapping objects. Subsequent versions—YOLOv2, YOLOv3, and YOLOv4 [10]—addressed these issues by improving detection accuracy for smaller objects and enhancing speed, making YOLO a practical choice for real-time object detection applications.

The rest of this report is structured as follows. Section 2 reviews the information on the data and the architecture of the machine learning model. Section 3 presents and explains various results and figures related to the training process. In Section 4, the contents are summarized and the conclusion of the report is provided.

2 Data Set Treatment and ML Architecture

2.1 Data Set Treatment

Roboflow is a platform designed for creating and managing image datasets, specifically useful for machine learning and computer vision projects. Users can upload image data, perform augmentation, and prepare data for model training. Roboflow supports various formats and provides tools to find and utilize high-quality datasets, enabling efficient data collection and maximizing machine learning model performance.

https://roboflow.com/

I chose Roboflow to take advantage of its benefits and convenience. Roboflow can be accessed through the URL provided above. The image below shows the screen after accessing one of the datasets returned from searching 'crack detection' on Roboflow. Through 'Popular Download Formats,' you can download the data in your preferred format, and when downloading, you can receive the data not only in zip/tar files but also via a URL with a key, which allows for easy use in Colab.

Below that is the 'Dataset split,' which shows how the data provider has organized the dataset. The data is categorized into Train, Test, and Valid sets, and you can easily understand the size and proportion of each set. For these reasons, after downloading the data from Roboflow, I am able to train YOLO without any additional data pre-processing.

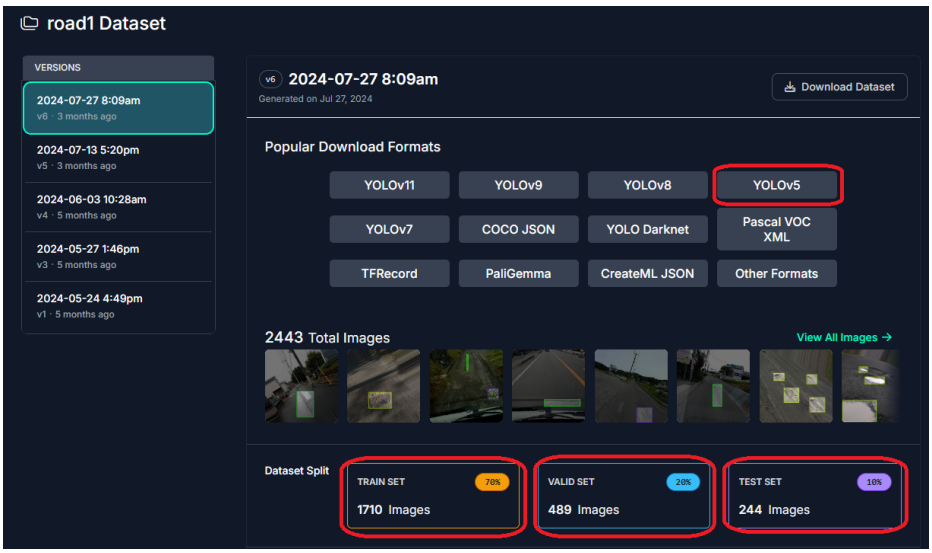

Figure 3: Dataset information screen on the Roboflow website

2.2 ML Architecture : YOLO Structure

YOLO (You Only Look Once) is a real-time object detection system that frames object detection as a single regression problem. The image is divided into a grid, and each grid cell is responsible for predicting bounding boxes and class probabilities.

The key mathematical concepts underlying YOLO are explained below:

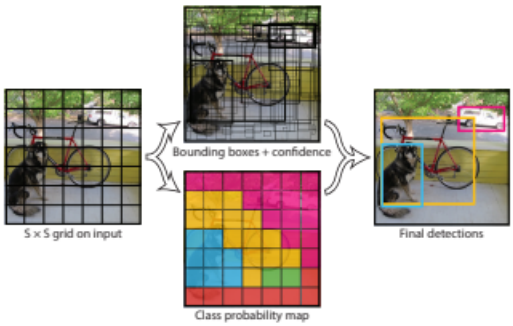

Figure 4: YOLO Model (Image taken from [9]).

2.2.1 Mathematical concepts

YOLO divides the input image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose center lies within the cell. Each cell predicts B bounding boxes, where each bounding box consists of:

$$(x, y, w, h)$$

where:

- x, y: the coordinates of the bounding box center relative to the grid cell,
- w, h: the width and height of the bounding box, normalized by the image width and height.

In addition to the bounding box, each cell predicts a confidence score C, which reflects the accuracy of the prediction and the probability of an object being present in the bounding box:

$$C = P(\text{Object}) \times \text{IOU}_{\text{pred}}^{\text{truth}}$$

where $P(\text{Object})$ is the probability of an object being present and $\text{IOU}_{\text{pred}}^{\text{truth}}$ is the Intersection over Union (IoU) between the predicted and ground truth bounding boxes. Each grid cell also predicts a set of class probabilities $P(C_i|\text{Object})$ for each of the C classes, given that an object is detected in the cell. The final prediction for each grid cell includes both the bounding box and the class probabilities, which are combined into a final output. YOLO's loss function is designed to optimize the accuracy of both bounding box predictions and class predictions. The total loss consists of three parts:

$$\text{Loss} = \lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 \right]$$

$$+\lambda_{\text{coord}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right]$$

$$+ \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{obj}} (C_i - \hat{C}_i)^2 + \lambda_{\text{noobj}} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbf{1}_{ij}^{\text{noobj}} (C_i - \hat{C}_i)^2$$

$$+ \sum_{i=0}^{S^2} \mathbf{1}_{i}^{\text{obj}} \sum_{c \in \text{classes}} (p_i(c) - \hat{p}_i(c))^2$$

where:

- $\mathbf{1}_{ij}^{\text{obj}}$ is an indicator function for whether the i-th grid cell contains an object.
- λ_{coord} and λ_{noobj} are hyperparameters that control the trade-off between coordinate and confidence loss.
- $p_i(c)$ is the predicted probability of class c for the i-th grid cell.

This loss function ensures that YOLO can jointly optimize both localization and classification tasks.

2.2.2 YOLO Architecture

The figure illustrates the architecture of the YOLO (You Only Look Once) model, which is designed for object detection. YOLO processes an image in a single forward pass, predicting both the location and class of objects simultaneously.

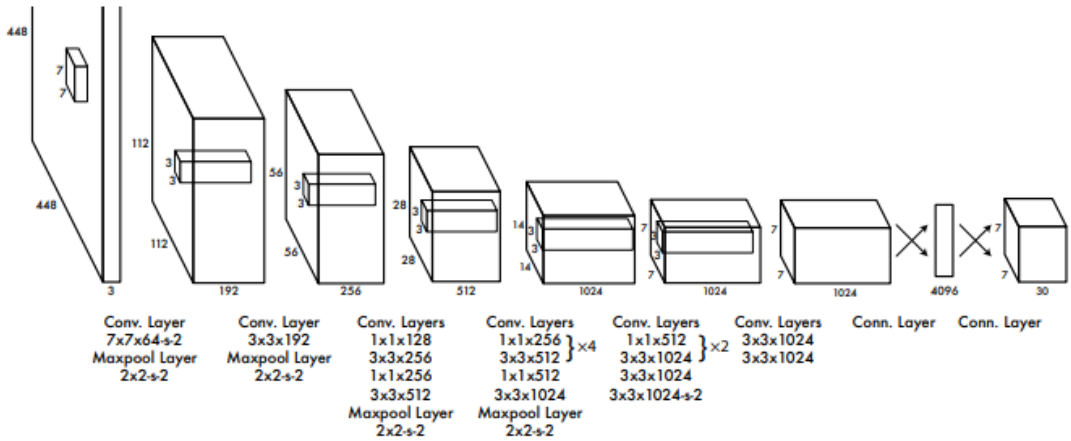

Figure 5: YOLO Architecture (Image taken from [9])

1. Input Size: The input to the model is an RGB image with a size of $448 \times 448 \times 3$. This provides a high-resolution image for YOLO to determine object locations within the image.

2. Conv Layer (7x7x64-2): The first layer is a convolutional layer with a 7x7 kernel and 64 filters. It operates with a stride of 2, which reduces the resolution of the input image, capturing essential features.

3. Maxpool Layer (2x2-2): A max-pooling layer of size 2x2 and stride 2 is applied to further reduce the spatial dimensions by half, while retaining the most prominent features of the image.

4. Conv Layers (3x3): Several convolutional layers with 3x3 filters follow, with increasing numbers of filters (192, 256, 512, 1024). These layers extract high-dimensional features from the input image and pass them to deeper layers for further processing.

5. 1x1 Conv Layers: 1x1 convolutional layers are used to reduce the dimensionality of the feature space and efficiently process the information from the preceding layers. These layers help in reducing computational cost without losing significant information.

6. Maxpool Layer (2x2-2): Additional max-pooling layers are applied throughout the network to further downsample the image. These layers reduce the input size while retaining critical information necessary for object detection.

7. Fully Connected Layers: At the end of the architecture, two fully connected layers are present, each with 4096 neurons. These layers are responsible for making the final predictions for object classes and bounding box coordinates.

In the final connected layer, the model simultaneously predicts the class of the object and the bounding box coordinates. YOLO simplifies object detection by processing the entire image in one go and predicting the object locations and classes in a single pass.

3 Results and Discussion

In this chapter, I will run YOLO using real data and examine the training results as well as the predicted data. Additionally, the chapter will include an evaluation of the training and provide additional comments. Sampled data from the training process will also be provided as examples.

3.1 Experiment Results

The following experiment was conducted based on the following settings

Parameter	Value
YOLO Model	yolov5x
Image Size	512
Batch Size	32
Epochs	100

Table 1: List of Hyperparameters

3.1.1 Loss Function

The loss curves visualize how the loss function values change during the training process.

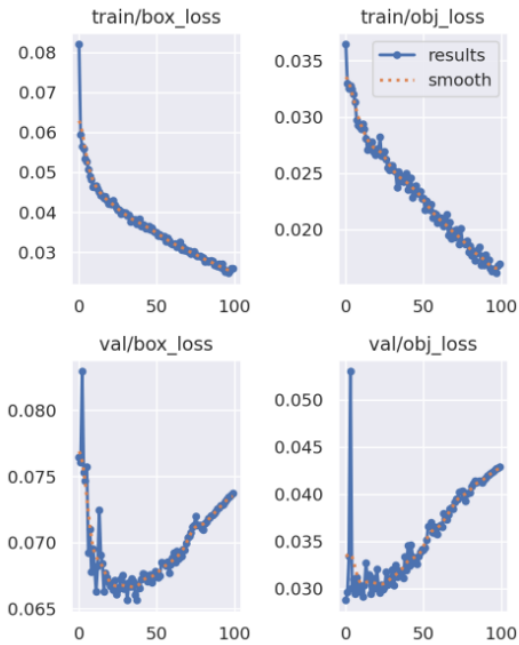

Figure 6: Loss function

- The **Train/Box Loss** (Top Left) represents the loss in predicting the coordinates of the bounding box. Throughout the training process, the train Box Loss steadily decreases, indicating that the model is gradually improving its ability to predict bounding box coordinates. By the time the training reaches 100 epochs, the Box Loss stabilizes around 0.03, showing that the model has been trained sufficiently.

- The **Train/Obj Loss** (Top Right) refers to the loss related to predicting the presence of an object within the image. Initially, the Obj Loss is relatively high, but it decreases sharply as the training progresses. Eventually, it stabilizes between 0.015 and 0.02, indicating that the model is improving its object detection capabilities as the number of epochs increases.

- The **Val/Box Loss** (Bottom Left) represents the bounding box prediction loss on the validation dataset. The validation Box Loss decreases initially, but after about 50 epochs, it starts to increase again. This pattern suggests that overfitting may be occurring after a certain number of epochs, where the model performs well on the training data but begins to lose performance on unseen validation data.

- The **Val/Obj Loss** (Bottom Right) refers to the object presence prediction loss on the validation dataset. The Obj Loss decreases sharply at first, but it starts to increase again after around 50 epochs, indicating that the model might be overfitting to the training data, leading to degraded performance on the validation set.

3.1.2 Precision-Confidence Curve

The graph shown is a Precision-Confidence Curve, which visually represents the relationship between Precision and Confidence.

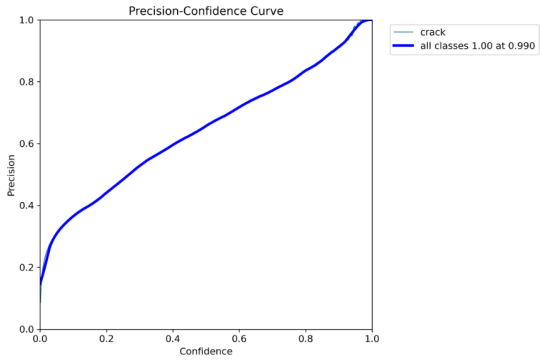

Figure 7: Precision-Confidence Curve

- **Initial State (0 - 0.2)**: At confidence values between 0 and 0.2, the precision starts from around 0.2 and slowly increases. This indicates that the model may make a significant number of incorrect predictions when the confidence is low.

- **Middle Range (0.2 - 0.8)**: As confidence increases from 0.2 to 0.8, the precision steadily improves. When the confidence exceeds 0.6, the precision remains above 0.6, showing that the model's predictions become more accurate as confidence grows. In this range, the model makes more accurate predictions with higher confidence.

- **Final Range (0.8 - 1.0)**: At confidence levels above 0.8, precision reaches approximately 0.9 and remains high. At the highest confidence level (Confidence = 1.0), the model's precision approaches 1.0, indicating that the model makes nearly flawless predictions when it is highly confident.

3.1.3 Precision-Recall Curve

This graph helps evaluate how well the model performs in terms of both accuracy (precision) and the ability to detect objects (recall).

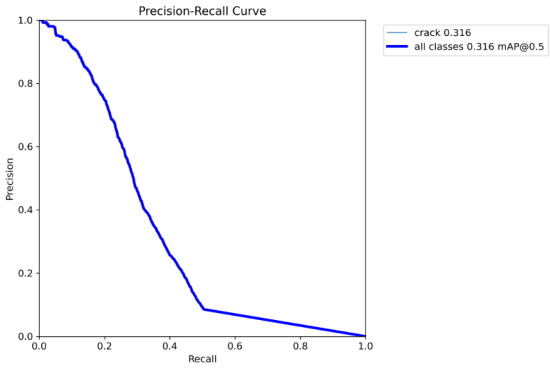

Figure 8: Enter Caption

- **Initial phase (0 - 0.2)**: The model starts with very high precision, meaning that it is making very cautious predictions. However, the recall is low in this region, indicating that the model is missing many objects.
- **Middle phase (0.2 - 0.6)**: As recall increases, precision drops sharply. This indicates that while the model is detecting more objects, the number of false positives is increasing as well.
- **Final phase (0.6 - 1.0)**: Once recall surpasses 0.6, the precision drops to almost zero. This suggests that as the model tries to capture most objects, its accuracy suffers significantly. In this range, the model is likely producing many false detections in its attempt to achieve high recall, sacrificing precision.

3.1.4 F1-Confidence Curve

F1-score represents the balance between precision and recall.

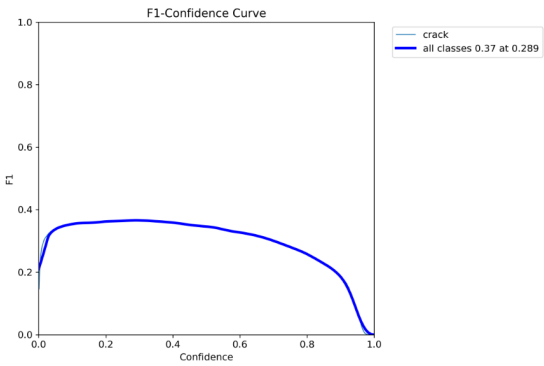

Figure 9: Enter Caption

The analysis of the graph is as follows:

- **Initial phase (0 - 0.2)**: In the confidence range between 0 and 0.2, the F1-score rises sharply. This suggests that, with low confidence, the model's performance is weak, but as confidence increases slightly, the model's performance improves quickly.
- **Middle phase (0.2 - 0.6)**: In this range, the F1-score stabilizes around 0.4. This indicates that the model shows a balanced performance in this confidence range, where its predictions are moderately confident and well-balanced between precision and recall. However, increasing the confidence further does not result in significant improvements in the F1-score.

- **Final phase (0.6 - 1.0)**: As the confidence exceeds 0.6, the F1-score gradually declines. Particularly, beyond a confidence value of 0.9, the F1-score drops sharply. This suggests that while the model is highly confident in its predictions at this stage, the balance between precision and recall may be compromised, leading to lower performance. The model may become overconfident, resulting in more incorrect predictions.

3.1.5 Object Detection Performance Evaluation

Here, we examine sample data to evaluate how well the bounding box captures the objects.

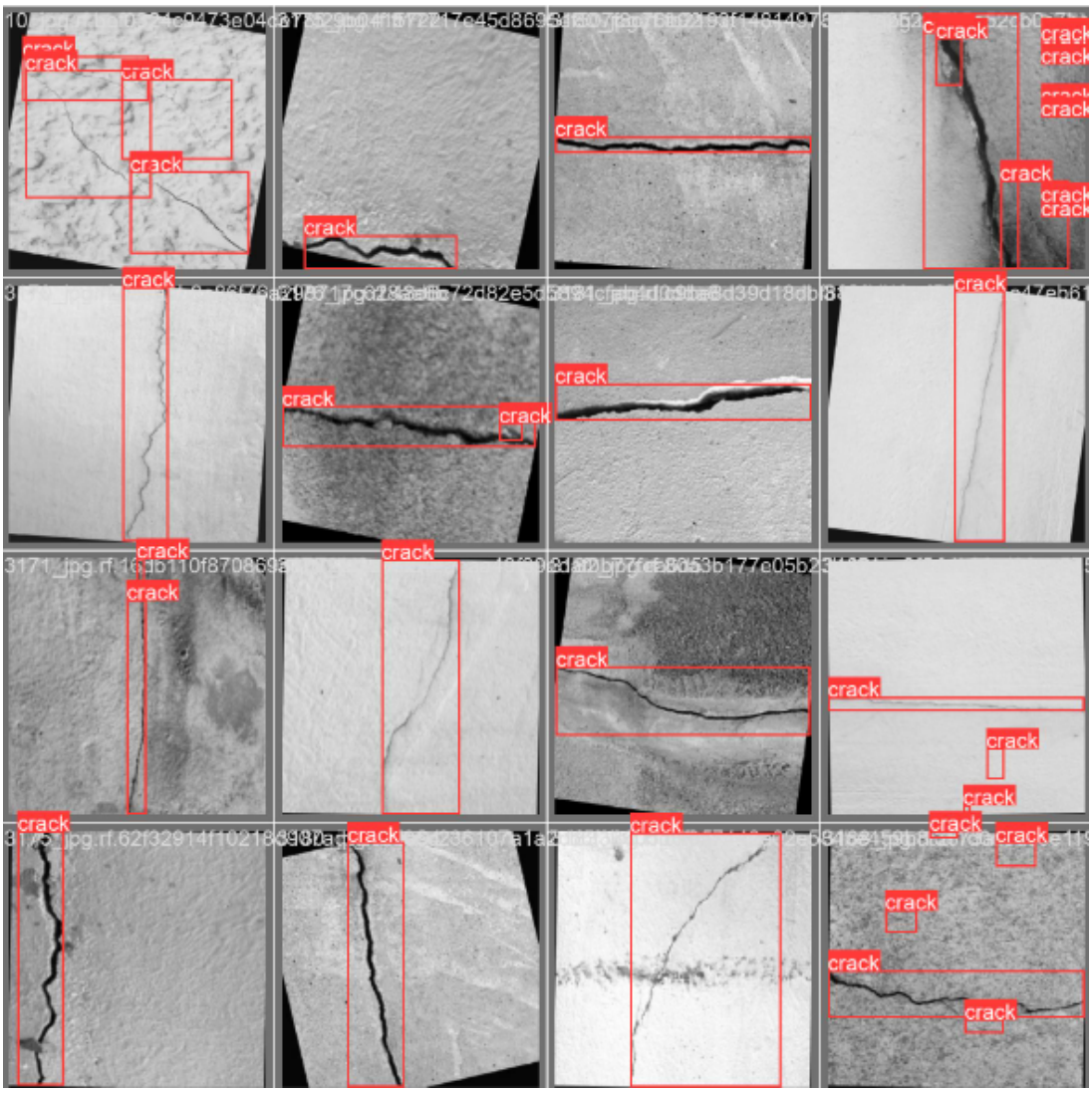

Figure 10: Samples of Crack Detection1

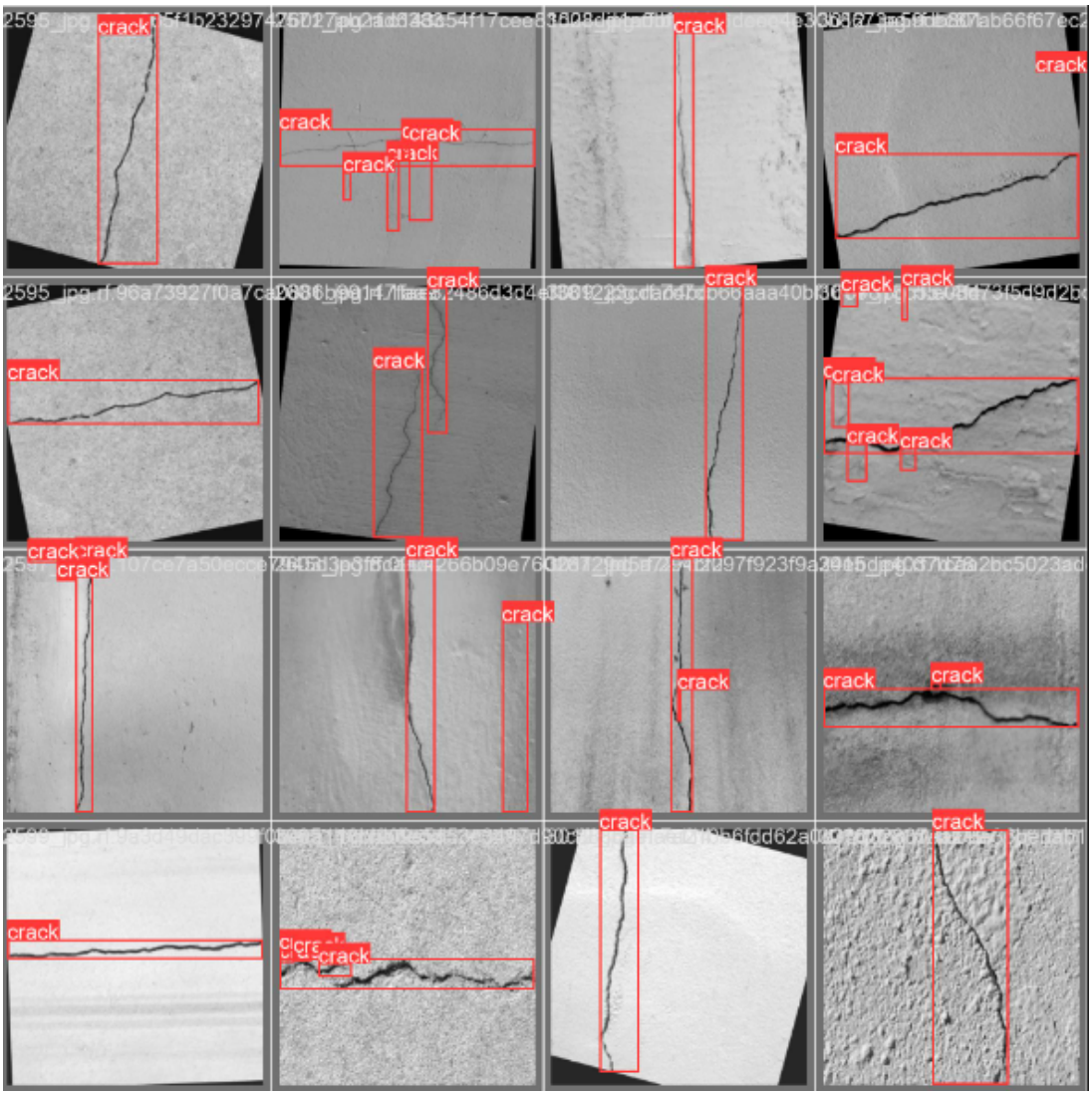

Figure 11: Samples of Crack Detection2

3.2 Discussion for Further Improvement

In this section, I focus on areas requiring further attention and potential improvements to enhance the model's performance.

3.2.1 Overfitting Concerns

Both **Val/Box Loss** and **Val/Obj Loss** demonstrate an increasing trend after around 50 epochs, suggesting that the model is beginning to overfit. While the training loss continues to decrease, the validation loss starts to rise, indicating that the model is becoming less effective at generalizing to unseen data. To mitigate this issue, techniques such as **early stopping** or the introduction of **regularization** methods (e.g., L2 regularization or dropout) should be considered. Early stopping will prevent over-training, while regularization will reduce model complexity and help avoid fitting noise in the training data.

3.2.2 Precision-Recall Trade-off

The **Precision-Recall Curve** indicates a sharp drop in precision as recall increases, particularly after a recall value of 0.6, where precision falls to near zero. This suggests that as the model tries to detect more objects (increasing recall), it introduces many false positives (reducing precision). Improving the balance between precision and recall is essential, and it may be necessary to adjust the **thresholds** for classification. Additionally, increasing the quantity and diversity of training data, especially for difficult-to-detect objects, could help the model improve its performance across both metrics.

3.2.3 Confidence Threshold Tuning

The **F1-Confidence Curve** shows a significant drop in the F1-score when the confidence exceeds 0.9. This suggests that although the model is highly confident in its predictions, this overconfidence may lead to an imbalance between precision and recall, resulting in a lower overall F1-score. Tuning the **confidence threshold** is a crucial next step. By carefully selecting an appropriate threshold, we can ensure that predictions remain balanced and avoid situations where the model is overly confident in inaccurate predictions. Validation on high-confidence predictions should be performed to assess how well the model generalizes in these cases.

3.2.4 Data Augmentation and Expansion

To address the issues of overfitting and the trade-off between precision and recall, data augmentation strategies should be explored. Techniques such as random cropping, rotations, and adjustments in lighting can diversify the training data, allowing the model to better generalize. Additionally, expanding the dataset with more varied and challenging examples will help reduce the precision-recall gap by enabling the model to handle more diverse scenarios.

3.2.5 Hyperparameter Tuning

Finally, further optimization of **hyperparameters**, such as the learning rate, batch size, and the number of epochs, should be explored. Fine-tuning these parameters can lead to better performance, allowing the model to converge more efficiently while avoiding overfitting or underfitting.

3.2.6 Regular Monitoring of Validation Metrics

Continuous monitoring of validation metrics, such as **mAP** and **F1-score**, throughout the training process will ensure that any signs of overfitting or degradation in performance are detected early. Implementing automated validation checks during training can help identify and address these issues promptly, contributing to the overall robustness of the model.

In conclusion, focusing on mitigating overfitting, improving the precision-recall balance, tuning the confidence threshold, and optimizing hyperparameters will significantly enhance the model's generalization capabilities and accuracy.

4 Conclusion

In this study, I explored the use of the YOLO object detection framework for identifying cracks in large water infrastructure facilities. Through experimentation, the model's performance was evaluated using several key metrics, including loss functions, precision-recall curves, and F1-score confidence curves.

The results demonstrate that the YOLO model can efficiently predict the presence of cracks in images, achieving high precision at elevated confidence thresholds. However, challenges such as overfitting in the validation set and a steep trade-off between precision and recall were identified, particularly when attempting to increase recall beyond certain limits.

Additionally, the sharp drop in F1-score for high confidence thresholds indicates that overconfidence in the model's predictions can result in imbalanced outcomes, compromising the overall performance. To address these issues, further refinements such as early stopping, regularization, confidence threshold tuning, and data augmentation should be considered.

In summary, while the model shows potential for real-time crack detection in water facilities, addressing the overfitting and balancing precision and recall are essential for further improving its reliability and robustness.

5 Final Remarks for the Further Application to Study

The results of this study indicate that the YOLO object detection model is a valuable tool for automating crack detection in water infrastructure. By incorporating further techniques such as data augmentation, regularization, and hyperparameter tuning, the model can be optimized for even greater accuracy and reliability.

For future applications, the following areas should be explored:

- **Model Transferability:** Investigating how well the trained model performs on different types of infrastructure or in different environmental conditions (e.g., lighting, weather) can provide insights into its generalizability.
- **Active Learning:** By incorporating active learning strategies, the model could iteratively improve by selecting the most informative data points for manual labeling and retraining.
- **Real-time Deployment:** Further research is required to optimize the model for real-time use, ensuring that the object detection is fast and efficient for large-scale deployment in monitoring systems.
- **Integration with Monitoring Systems:** Finally, the integration of this model with existing infrastructure monitoring systems could provide early warning signals for potential failures, thus improving maintenance and safety protocols.

In conclusion, this study shows that AI-driven solutions like YOLO can significantly enhance the efficiency and accuracy of crack detection in critical infrastructure, paving the way for improved monitoring and safety management.

References

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," Proceedings of the IEEE, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] A. Krizhevsky, I. Sutskever, and G. Hinton, "ImageNet classification with deep convolutional neural networks," in Advances in Neural Information Processing Systems (NeurIPS), 2012, pp. 1097-1105.

[3] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," arXiv preprint arXiv:1409.1556, 2014.

[4] C. Szegedy, W. Liu, Y. Jia, et al., "Going deeper with convolutions," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2015, pp. 1-9.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[6] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2014, pp. 580-587.

[7] R. Girshick, "Fast R-CNN," in Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2015, pp. 1440-1448.

[8] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," in Advances in Neural Information Processing Systems (NeurIPS), 2015, pp. 91-99.

[9] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.

[10] J. Redmon and A. Farhadi, "YOLO9000: Better, faster, stronger," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 6517-6525.